Compressing Sequences in the Latent Embedding Space: *K*-Token Merging for Large Language Models

Zihao Xu[◇*], John Harvill[♡*], Ziwei Fan[♠], Yizhou Sun[♣†], Hao Ding[♠], Hao Wang[◇△]

[◇]Rutgers University [♠]AWS AI Labs [♣]Amazon [♡]Mistral AI
[†]University of California, Los Angeles [△]University of Illinois Urbana-Champaign













* Work done at AWS AI Labs

MOTIVATION

Why do we need token compression?

Agents
long tool-call histories

RAG
many retrieved documents

Many-shot ICL
hundreds of examples up front

→ **Longer and longer contexts** →

Attention cost $\propto n^2$
2× longer ⇒ 4× compute

Token compression:
Save both **inference time** and **GPU memory**

MOTIVATION

Do existing compression methods solve it?

> Not fully

Hard prompt compression

- Drop or summarize tokens
- Fails on **information-dense** tasks

Soft prompt compression

- Learn new meta-tokens
- **Limited** length reduction

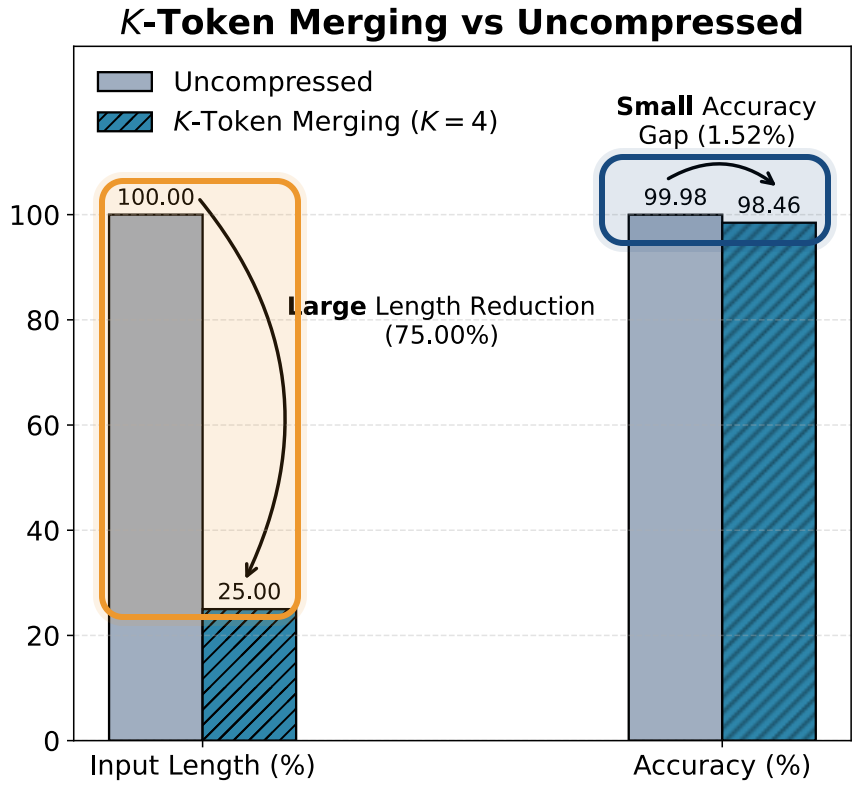
Problem: only work at the **token level**

Overlooked: the **latent embedding space**

MAIN RESULT

How much can we compress in embedding space?

> Up to **75% shorter input** with minimal performance drop



Textualized Tree, $K = 4$

75%

input length reduction

1.52 pts

accuracy drop

KEY OBSERVATION

Where does this magic come from?

> Token embeddings are **highly redundant**

bits used per token embedding

28,672

896×32 (Qwen2.5-0.5B)

bits needed to identify a token

≈ 18

$\log_2 151{,}936$

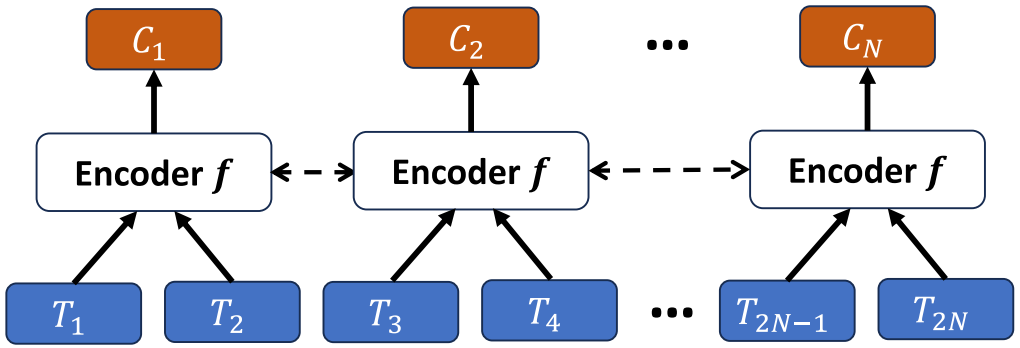
K tokens need only $\approx 18K$ bits $\Rightarrow$ **one embedding can hold several tokens**

METHOD

How do we exploit the redundancy?

> Merge every ***K* token embeddings into one**

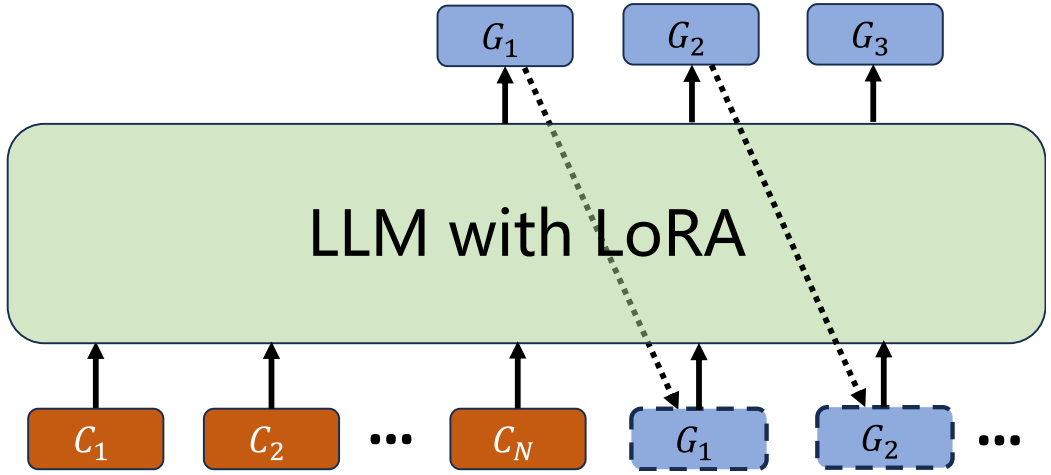
Prefill



$$C_i = f\big(\mathrm{Emb}(T_{iK+1}), \ldots, \mathrm{Emb}(T_{iK+K})\big)$$

Example: $K = 2$ · Encoder f: small MLP (~50 MB)

Generation



Outputs original tokens

Decoded autoregressively

METHOD

How do we train it?

> Co-train **encoder + LoRA**; loss on **generated tokens**

$$X = (\underbrace{C_1, \ldots, C_N}_{\text{compressed input}}, \underbrace{G_1, \ldots, G_M}_{\text{generated tokens}})$$

$$\mathcal{L}(\theta, f) = -\sum_{t \in \mathcal{U}} \log p_\theta(X_t \mid X_{<t}), \qquad \mathcal{U} = \{N+1, \ldots, N+M\}$$

θ: LoRA adapter · f: encoder · $\mathcal{U}$: generated-token indices

EXPERIMENTS

What do we evaluate on?

> Three tasks, each testing a different ability

Textualized Tree

information preservation

accuracy

Amazon Reviews

natural language

accuracy

CommitPackFT

code generation

perplexity

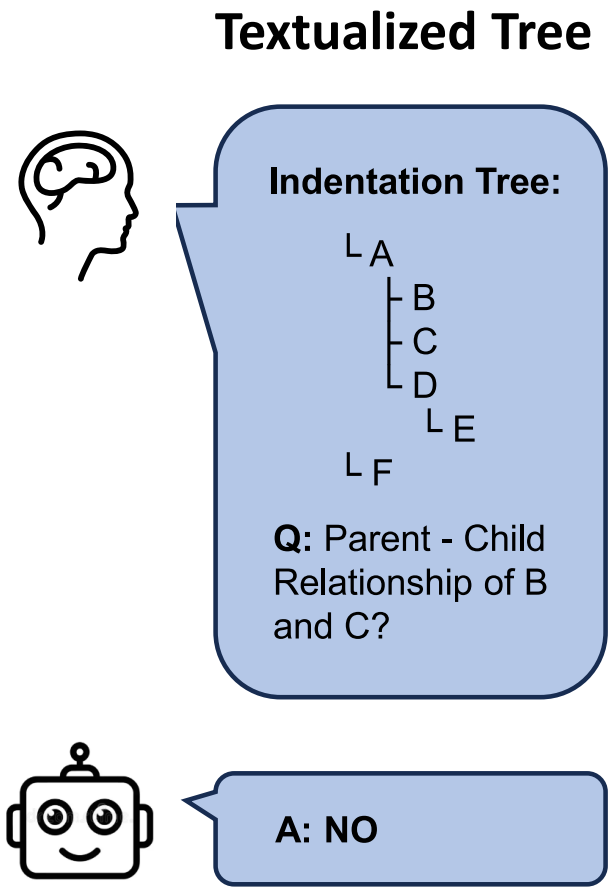
Baselines

- **Hard compression**: SelectiveContext, LLMLingua2
- **Soft compression**: LTSC

Setup

- Qwen2.5-0.5B (+ Qwen2.5-Coder 7B)
- Performance vs. length reduction

Textualized Tree: is information preserved?



Task

Given an indentation tree, is node B the parent of node C?

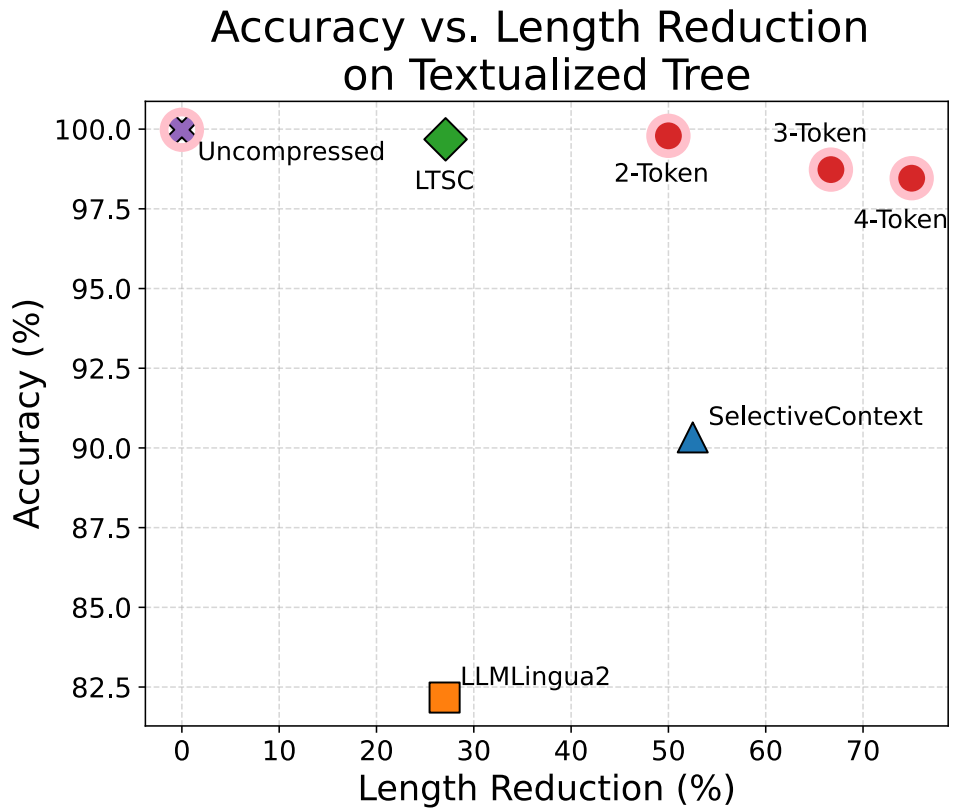
Why it is hard

Any pair of nodes can be asked about, so the compressed input must keep the *full* structure

> Tests **information preservation**

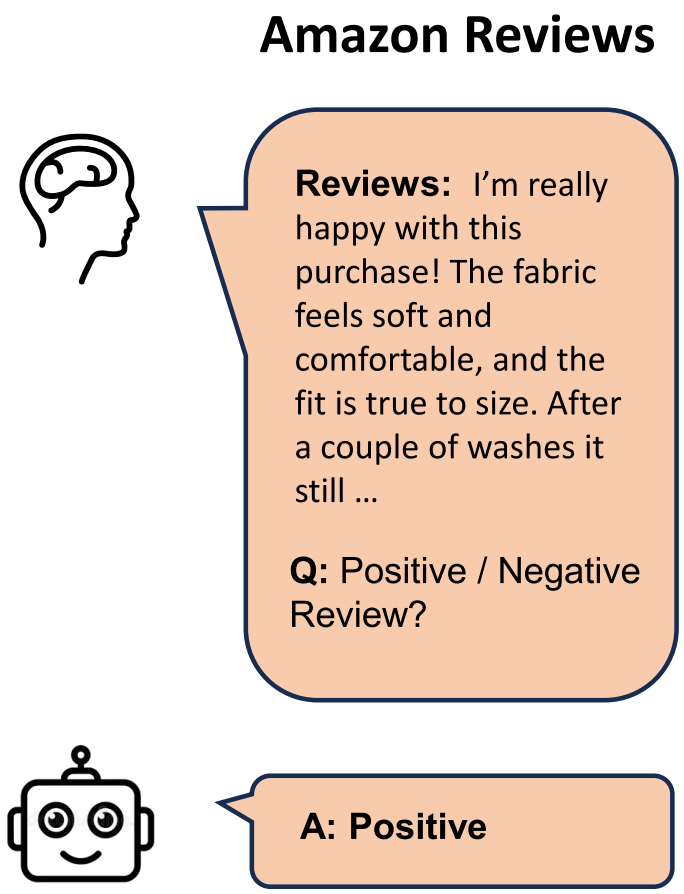
Textualized Tree: hard compression loses structure

Hard compression drops key tokens; KTM keeps **98.46%** at **75%** reduction



Method		Reduction ↑	Accuracy ↑
Uncompressed		0.0%	99.98
HARD	SelectiveContext	52.5%	**90.33**
	LLMLingua2	27.0%	**82.17**
SOFT	LTSC	27.1%	99.68
OURS	**2-Token**	50.0%	99.79
	3-Token	66.7%	98.73
	4-Token	**75.0%**	98.46

Amazon Reviews: does it work on natural language?



Task

Classify a Fashion-category review as positive or negative (rating > 3.5 is positive)

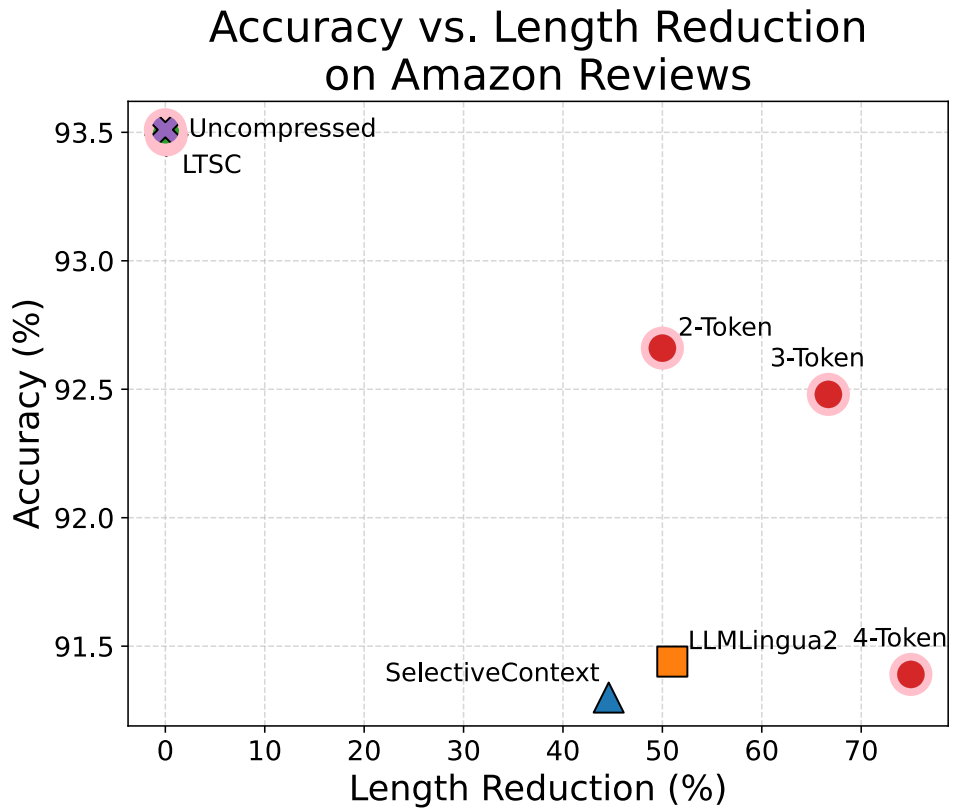
Why it is hard

Real-world text with few exactly repeated patterns to exploit

> Tests **natural language**

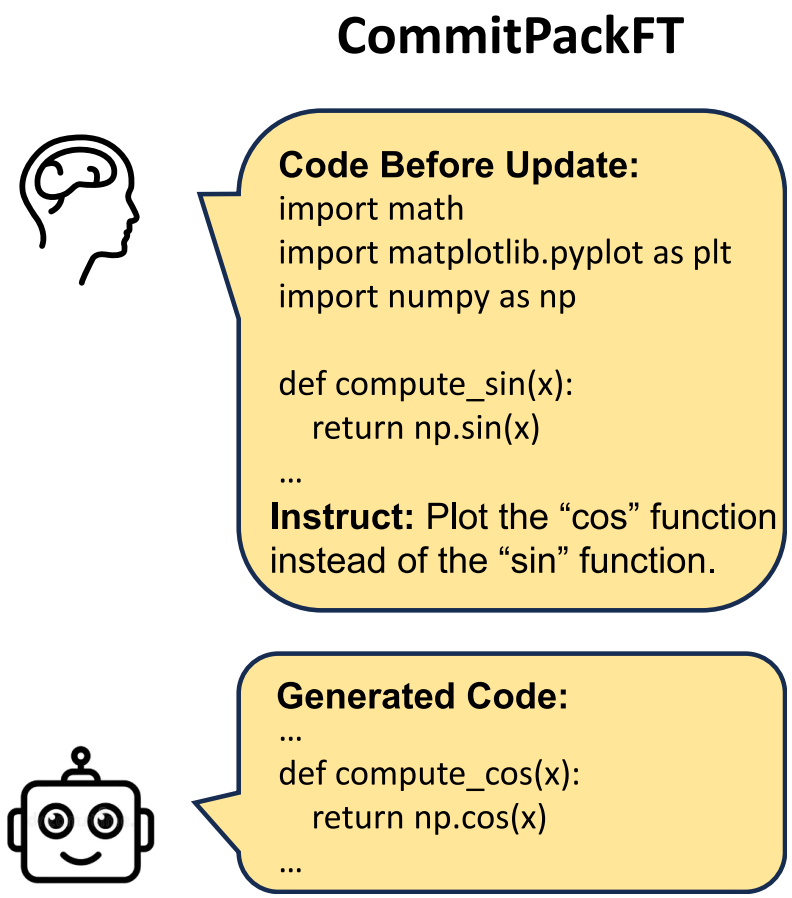
Amazon Reviews: soft compression can hardly compress

> **Soft** compression reduces length by only **0.1%**; KTM halves it for **−0.85 pts**



Method		Reduction ↑	Accuracy ↑
Uncompressed		0.0%	93.51
HARD	SelectiveContext	44.6%	91.30
	LLMLingua2	51.0%	91.44
SOFT	LTSC	**0.1%**	93.49
OURS	**2-Token**	50.0%	92.66
	3-Token	66.7%	92.48
	4-Token	**75.0%**	91.39

CommitPackFT: does it work on code generation?



Task

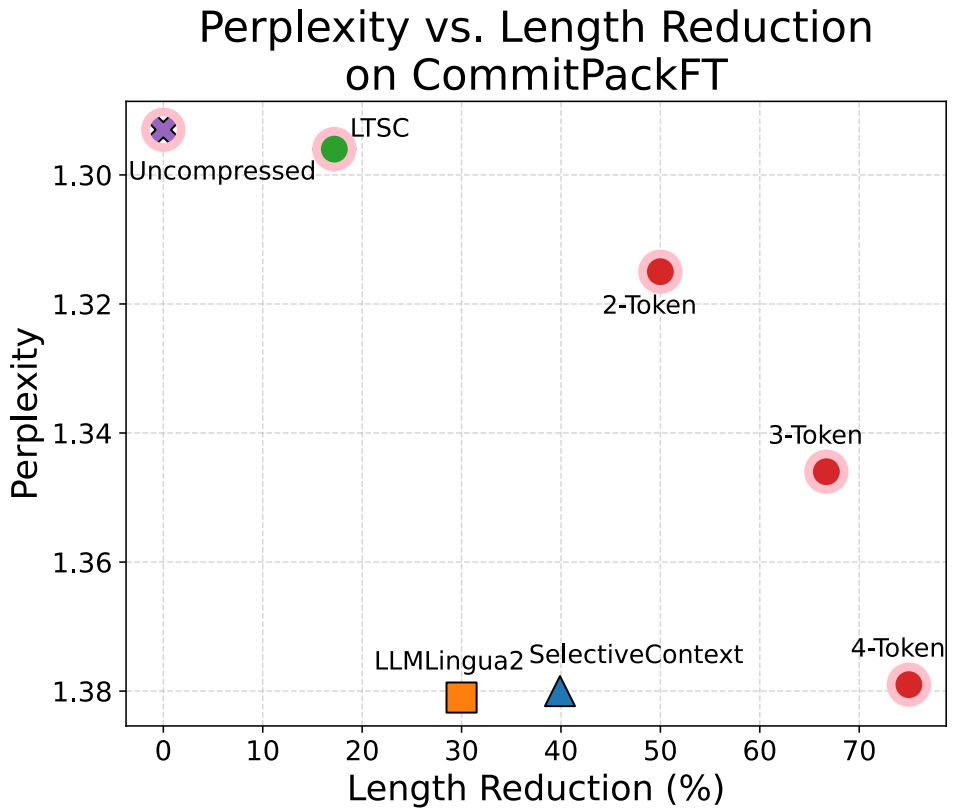
Given the code before a commit and the commit message, generate the updated code

Why it is hard

Information-dense: dropping any token can break the program

> Tests **code generation**

CommitPackFT: best trade-off on code generation



Method		Reduction ↑	Perplexity ↓
Uncompressed		0.0%	1.293
HARD	SelectiveContext	39.9%	1.380
	LLMLingua2	30.0%	1.381
SOFT	LTSC	17.2%	1.296
OURS	**2-Token**	50.0%	1.315
	3-Token	66.7%	1.346
	4-Token	**75.0%**	1.379

Does compression lose information?

INPUT · COMPRESSED (K = 2)

```
import math
import matplotlib.pyplot as plt
import numpy as np

# Define the function
def compute_sin(x):
    return np.sin(x)

# Generate input values
x_vals = np.linspace(-2 * np.pi, 2 * np.pi, 500)
y_vals = compute_sin(x_vals)

# Plotting
plt.figure(figsize=(8, 4))
plt.plot(x_vals, y_vals, label='sin(x)')
plt.show()
```

Instruction “plot the ‘cos’ function at the same time”

OUTPUT · GENERATED

```
...
# Generate input values
x_vals = np.linspace(-2 * np.pi, 2 * np.pi, 500)
y_vals = compute_sin(x_vals)

# Plotting
plt.figure(figsize=(8, 4))
plt.plot(x_vals, y_vals, label='sin(x)')
plt.show()
# Plotting the "cos" function
plt.figure(figsize=(8, 4))
plt.plot(x_vals, np.cos(x_vals), label='cos(x)')
plt.legend()
plt.show()
```

No: it **follows** the instruction and **reuses** `x_vals`

Does it hold on a larger model, in real wall-clock time?

−20.9%
latency

+27.8%
throughput

+1.1%
perplexity

Qwen2.5-Coder 7B · CommitPackFT · 2-Token Merging

Metric	No compression	2-Token Merge
Perplexity ↓	**1.263**	1.277
Latency (s) ↓	0.789	**0.624**
Throughput (samples/s) ↑	5.271	**6.736**
GPU peak memory (MB) ↓ *	**35,983**	38,349

* Batch size 2: memory is dominated by model weights + encoder; token savings grow with batch size.

CONCLUSION

Takeaway:

> Compress in the **latent embedding space**, not just in token space

$K \rightarrow 1$

tokens per embedding
(encoder + LoRA)

up to

75%

shorter input
Pareto-optimal on 3 tasks

–20.9%

latency
on a 7B model

Thank you!



Code

github.com/shsjxzh/K-Token-Merging

Paper

arxiv.org/abs/2604.15153

Contact

zihaokxu@amazon.com

Why not just add *K*-grams to the vocabulary?

	K-gram vocabulary	*K*-Token Merging (ours)
Modifies	input and output	**input only**
Vocabulary size	explodes (4-grams ≈ 29×)	**unchanged**
Rare tail tokens	many, hard to train	**none**
Unseen *K*-grams	no embedding	**handled by encoder**
Large *K*	infeasible	**same design**

What does each component contribute?

> The encoder brings **quality**; the shorter sequence brings **speed**

Metric	KTM (ours)	Mean pooling only	No compression
Perplexity ↓	1.447	1.453	**1.364**
Latency (s) ↓	0.358	**0.353**	0.546
Throughput (samples/s) ↑	17.792	**18.038**	11.377
GPU peak memory (MB) ↓	9,552	**9,232**	13,006

- Learnable residual vs. mean pooling: perplexity **1.453 → 1.447**
- vs. no compression: **−34.4%** latency, **+56.4%** throughput, **−26.6%** memory

CommitPackFT, 10 epochs each. “No compression” uses a K→K encoder with a matched parameter budget of our method.

How do we initialize the encoder?

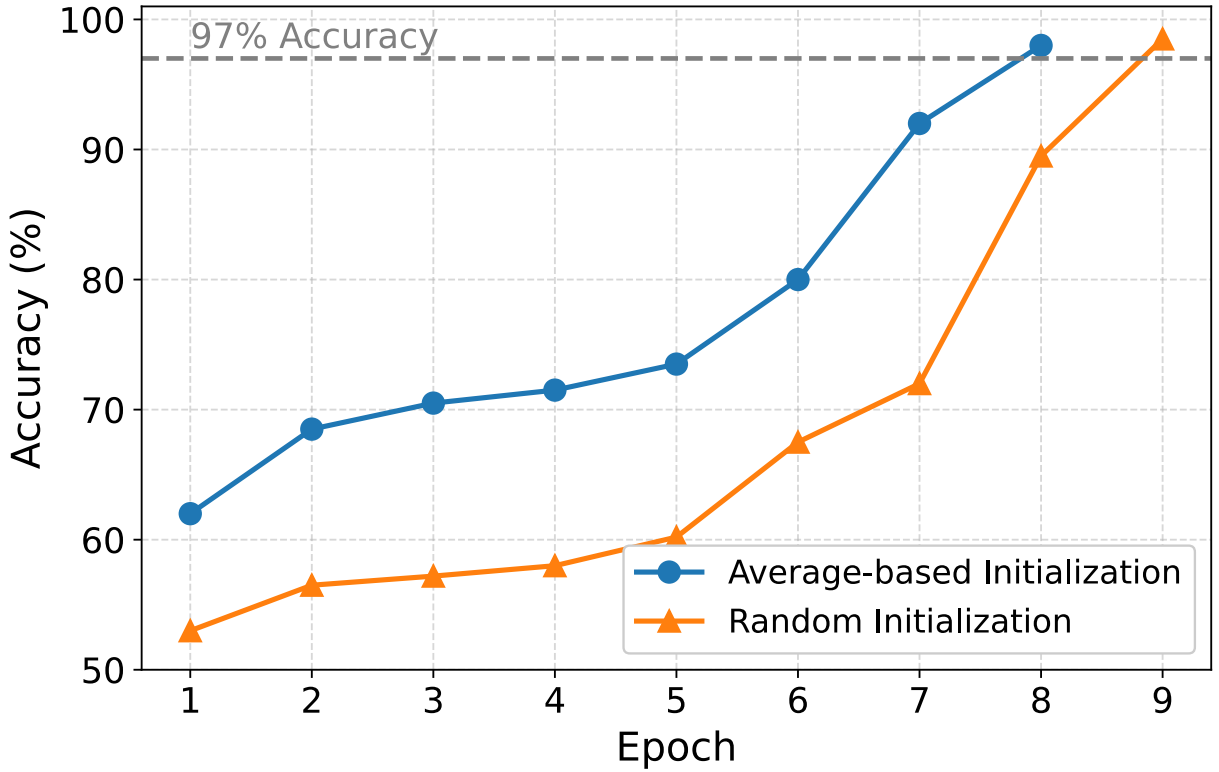
> Start from the **average** of the *K* embeddings

$$f(e_1, \ldots, e_K) = \underbrace{\frac{1}{K}\sum_j e_j}_{\text{mean pooling}} + \underbrace{\mathrm{MLP}\big([e_1; \ldots; e_K]\big)}_{\approx 0 \text{ at init}}$$

```
# x: (batch, embedding_dim, K)
mean   = x.mean(dim=-1)
x_flat = x.reshape(batch, embedding_dim * K)
output = mean + self.net(x_flat)     # net initialized near 0
```

Stable, semantic starting point · Faster convergence

Does average initialization help?



4-Token Merging · 5-node trees

8 epochs
average-based init

9 epochs
random init

Average-based init:
1 epoch faster